LINUX INTERVIEW PREPARATION GUIDE

CHAPTER

01

Introduction to Linux

Interview-focused notes for DevOps, SysAdmin and Cloud Engineers (1–5 years of experience)

IN THIS CHAPTER

01 What Is Linux?
02 Linux vs Unix
03 The Linux Kernel
04 Linux Architecture
05 Kernel Space vs User Space
06 System Calls
07 The Shell
08 Command Execution Flow
09 Process, Thread and Daemon
10 Distributions and GNU/Linux
11 Important Linux Concepts
12 Interview Q&A
13 Chapter Takeaways

Ravi Tomar
Linux · DevOps · Cloud Infrastructure

1 What Is Linux?

Definition (Interview-Oriented)

Linux is an open-source, Unix-like operating system kernel that manages hardware resources and provides an environment for applications to run.

In everyday usage, the term **Linux** generally refers to a complete operating system built around the Linux kernel, along with system utilities, libraries, package managers, and applications.

Linux is widely used in:

- Web servers and application hosting.
- Cloud infrastructure such as AWS, Azure and GCP.
- Docker containers and Kubernetes.
- CI/CD pipelines and automation.
- Networking devices and embedded systems.

Why Do We Need Linux?

A computer contains hardware such as CPU, RAM, storage and network interfaces. Applications need a way to use these resources without directly controlling the hardware.

An operating system provides this functionality.

For example, when you run a web application:

- The CPU executes application instructions.
- RAM stores application data.
- Storage holds application files and logs.
- The network interface handles incoming and outgoing requests.

Linux manages these resources and allows multiple applications to operate efficiently and securely.

Why Is Linux Popular in DevOps?

- **Open Source:** Its source code is publicly available, allowing inspection and customization.
- **Stability:** Commonly used for long-running production workloads.
- **Automation:** Provides powerful command-line tools and scripting capabilities.
- **Security:** Supports users, permissions, access controls and security mechanisms.
- **Cloud Support:** Widely supported by cloud providers.
- **Containerization:** Docker and Kubernetes rely heavily on Linux kernel features.

INTERVIEW QUESTION

Why is Linux preferred over Windows for many DevOps workloads?

Linux is widely preferred because of its command-line capabilities, automation support, resource efficiency, open-source ecosystem, and strong support for cloud-native technologies. However, Windows is also widely used in DevOps, particularly for .NET and Windows-based applications.

2 Linux vs Unix

Linux was developed using Unix principles, but it is not the original Unix operating system.

What Is Unix?

Unix is a family of operating systems originally developed at AT&T Bell Labs in the 1970s. It introduced many concepts that influenced modern operating systems, including hierarchical filesystems, multiuser environments and command-line utilities.

Examples include AIX, HP-UX and Solaris.

Key Differences

Feature	Unix	Linux
Origin	Developed at Bell Labs	Created by Linus Torvalds
First release	1970s	1991
Source code	Varies; often proprietary	Open-source kernel
Customization	Depends on vendor	Highly customizable
Usage	Enterprise and legacy systems	Servers, cloud, containers and desktops
Examples	AIX, HP-UX, Solaris	Ubuntu, Debian, RHEL

INTERVIEW NOTE

Linux is Unix-like, but Linux is not the original Unix operating system.

3 What Is the Linux Kernel?

Definition

The Linux kernel is the core component of the operating system. It manages hardware resources and provides essential services that applications need to run.

Think of the kernel as the manager of the entire system.

Major Responsibilities of the Kernel

Responsibility	What It Does
Process Management	Manages running processes and CPU scheduling.
Memory Management	Allocates and manages RAM and virtual memory.
Device Management	Communicates with hardware through device drivers.
Filesystem Management	Provides access to files and storage devices.
Networking	Handles network communication and protocols.
Security	Enforces access controls and isolation mechanisms.

Real-Life Example

Suppose you run an Nginx web server on a Linux machine.

- Nginx requests memory to run.
- The kernel manages memory allocation.
- Nginx listens for network connections.
- The kernel handles network communication.
- Nginx reads website files from storage through kernel-managed filesystem operations.

Applications do not directly manage physical hardware. They request services from the kernel.

> INTERVIEW QUESTION
>
> **What happens if the Linux kernel stops working?**
>
> A critical kernel failure can cause a kernel panic, making the operating system unable to continue normal operation. The machine may become unresponsive or reboot, depending on its configuration.

4 Linux Architecture

Linux can be understood through four major layers.

Basic Architecture

```
+----------------------------------+
|        User / Applications       |
|   Nginx, Docker, Python, Java    |
+----------------------------------+
|        Shell / System Tools      |
|       Bash, SSH, Core Utilities  |
+----------------------------------+
|           Linux Kernel           |
| Process | Memory | Network | FS  |
+----------------------------------+
|             Hardware             |
|       CPU | RAM | Disk | NIC     |
+----------------------------------+
```

Architecture Explained

1. Hardware

Physical components of the computer. Examples: CPU, RAM, hard disk / SSD, network interface card.

2. Kernel

Manages hardware resources and provides services to applications.

3. Shell and System Utilities

Provide interfaces and tools to interact with the operating system. Examples: Bash, SSH, `ls`, `cp`, `grep`.

4. Applications and Users

Applications use operating system services to perform their work. Examples: Nginx, Docker, Jenkins, Python applications.

> **IMPORTANT CONCEPT**
>
> The shell is not the kernel. It is an interface through which users interact with the operating system.

5 Kernel Space vs User Space

Linux separates execution into two main areas: User Space and Kernel Space.

User Space

User Space is where normal applications and user-level processes execute.

Examples: Nginx, Docker CLI, Python scripts, Bash, Jenkins agents.

Applications running in User Space do not have unrestricted access to hardware or kernel memory.

Kernel Space

Kernel Space is where the Linux kernel executes with privileged access to system resources. It handles:

- CPU scheduling.
- Memory management.
- Hardware communication.
- Filesystem operations.
- Network operations.

Why Is This Separation Important?

- **Security:** Applications cannot directly access protected kernel memory.
- **Stability:** A normal application failure generally does not crash the entire operating system.
- **Isolation:** Applications operate with controlled permissions.
- **Resource Management:** The kernel controls how applications access system resources.

Real-Life Example

Suppose a Python application needs to read a file.

1. The application runs in User Space.
2. It requests file access through a system call.
3. The kernel checks permissions and handles the filesystem operation.
4. The result is returned to the application.

INTERVIEW QUESTION

Can a user-space application directly access hardware?

Normally, applications access hardware through kernel-provided interfaces and system calls rather than directly accessing physical hardware.

6 What Is a System Call?

Definition

A system call is a mechanism through which a user-space application requests a service from the Linux kernel.

Applications cannot perform privileged kernel operations directly, so they use system calls.

Common System Calls

System Call	Purpose
`open()`	Opens a file.
`read()`	Reads data.
`write()`	Writes data.
`close()`	Closes a file descriptor.
`fork()`	Creates a new process.
`execve()`	Executes a program.
`socket()`	Creates a network socket.

Real-Life Example

When an application reads a configuration file:

1. The application requests the file to be opened.
2. The kernel checks permissions.
3. The kernel provides access to the file.
4. The application reads the required data.

This interaction happens through system calls.

INTERVIEW QUESTION

What is the difference between a system call and a shell command?

A shell command is an instruction entered by a user. A system call is a low-level interface used by applications to request services from the kernel.

For example, `cat config.txt` is a shell command, while `open()` and `read()` are system calls used during file access.

7 What Is a Shell?

Definition

A shell is a command interpreter that accepts user commands, interprets them and executes the requested operations.

It provides an interface between the user and the operating system.

Common Linux Shells

Shell	Description
Bash	Bourne Again Shell; widely used in Linux.
sh	Traditional Unix shell interface. On Ubuntu, `/bin/sh` points to `dash`, not Bash.
Zsh	An interactive shell with additional customization.
Fish	A user-friendly interactive shell.

How Does a Shell Work?

```
User enters a command
         |
         v
Shell reads and interprets it
         |
         v
Finds and executes the command
         |
         v
Kernel provides required services
         |
         v
Output is displayed
```

Important: Shell vs Terminal

These two terms are often confused.

Terminal	Shell
Provides an interface to interact with the system.	Interprets and executes commands.
Displays input and output.	Processes commands.
Example: GNOME Terminal, Windows Terminal	Example: Bash, Zsh

Real-Life Example

When you open an Ubuntu terminal and execute:

```
ls -l
```

The terminal displays the interface, while Bash interprets the command and executes it.

> INTERVIEW QUESTION
>
> **Is Bash the Linux kernel?**
>
> No. Bash is a shell that runs in User Space. The Linux kernel is responsible for managing system resources and providing operating system services.

8 What Happens When You Execute a Linux Command?

Let's understand what happens when you execute:

```
ls -l /var/log
```

Execution Flow

Step 1 Command Input

You enter the command in the terminal.

Step 2 Shell Interpretation

The shell identifies the command, its arguments and options.

Step 3 Command Lookup

The shell first checks whether the command is an alias, a shell function or a shell built-in (such as `cd` or `echo`). If it is none of these, the shell searches the directories listed in the PATH environment variable.

```
echo $PATH
```

If the command is not found in any PATH directory, the shell returns `command not found`.

Step 4 Process Creation

The shell creates a child process using the `fork()` system call. The child process then loads the `ls` program using the `execve()` system call. The shell waits for the child process to finish.

Step 5 Kernel Interaction

The program requests filesystem access through system calls.

Step 6 Output Generation

The program reads the directory information and generates the output.

Step 7 **Output Display**

The output is returned to the terminal and displayed to the user.

INTERVIEW NOTE

`fork()` creates a copy of the shell process. `execve()` replaces that copy with the new program.

Practical Example: Find the Location of a Command

```
which ls
```

Another useful command:

```
type ls
```

`type` can also identify shell built-ins, aliases and functions.

INTERVIEW QUESTION

What happens when you execute a Linux command?

The shell interprets the command, locates the executable, starts its execution and interacts with the kernel through system calls when operating system services are required. The resulting output is then displayed in the terminal.

9 Process, Thread and Daemon

What Is a Process?

A process is a running instance of a program. Each process has its own PID, memory space and resources.

For example, the `nginx` binary stored on disk is a program. When it starts running, it becomes a process.

What Is a Thread?

A thread is a smaller unit of execution inside a process. Threads in the same process share the same memory and resources.

Real-Life Example

A Java application server such as Tomcat runs as one process. It uses many threads to handle multiple user requests at the same time.

Process vs Thread

Feature	Process	Thread
Memory	Has its own memory space	Shares memory with its process
Creation cost	Higher	Lower
Communication	Needs IPC (pipes, sockets)	Shares data directly
Failure impact	One crashed process does not affect others	One crashed thread can crash the whole process

Useful Commands

```
# Show all threads
ps -eLf

# Show threads inside top
top -H
```

What Is a Daemon?

A daemon is a background process that runs without a terminal and usually starts at boot through systemd.

Daemon names often end with the letter "d". Examples:

- `sshd` (SSH server)
- `crond` (cron scheduler)
- `dockerd` (Docker engine)
- `systemd-journald` (logging)

INTERVIEW QUESTION

What is the difference between a process and a thread?

A process is an independent running program with its own memory. A thread is an execution unit inside a process that shares the process's memory. Threads are lighter, but a crash in one thread can bring down the whole process.

INTERVIEW NOTE

Inside the Linux kernel, both processes and threads are represented as **tasks** and are scheduled in the same way.

10 Linux Distributions and GNU/Linux

What Is a Linux Distribution?

A Linux distribution is a complete operating system built around the Linux kernel, combined with system utilities, libraries, package management tools and other software.

The kernel alone is not everything required for a typical usable Linux environment. A distribution packages these components together to provide a working operating system.

What Is GNU/Linux?

GNU provides many essential user-space utilities, libraries and development tools used in Linux operating systems.

The Linux kernel works together with these and other software components to form a complete operating system. This is why the term GNU/Linux is sometimes used.

Popular Linux Distributions

Distribution	Common Usage
Ubuntu	Servers, cloud and development.
Debian	Stable servers and general-purpose systems.
RHEL	Enterprise production environments.
Amazon Linux	AWS EC2 workloads.
Rocky Linux	Enterprise-compatible Linux environments.
Fedora	Newer technologies and development.

Why Do Different Distributions Exist?

Different distributions target different requirements:

- Enterprise stability.
- Security and long-term support.

- Cloud compatibility.
- Development and testing.
- Community-driven customization.

> INTERVIEW QUESTION
>
> **What is the difference between Linux and Ubuntu?**
>
> Linux refers to the kernel, although it is commonly used to describe the broader operating system family. Ubuntu is a Linux distribution that includes the Linux kernel, system utilities, package management and other software.

11 Important Linux Concepts to Know

These are a few foundational terms that you will frequently encounter while working with Linux servers.

Term	Meaning
CLI	Command Line Interface used to interact with a system through commands.
GUI	Graphical User Interface used to interact through visual elements.
Root	The superuser account with UID 0.
PID	Process ID assigned to a running process.
Daemon	A background service or process that generally runs without direct user interaction.
Environment Variable	A named value available to processes, often used for configuration.
PATH	Environment variable containing directories searched for executable commands.
Filesystem	The structure and rules used to organize and access files on storage.
Swap	Disk space used to support virtual memory.
Load Average	Average number of processes running, waiting for CPU, or waiting for disk I/O over the last 1, 5 and 15 minutes. Compare it with the number of CPU cores (`nproc`).

Check Your Linux Environment

Run these commands to identify basic system information:

```
# Display kernel information
uname -a

# Display kernel version
uname -r

# Display Linux distribution details
cat /etc/os-release

# Display current username
whoami

# Display current working directory
pwd

# Display system hostname
hostname

# Display current shell
echo $SHELL

# Display PATH environment variable
echo $PATH
```

12 Interview Questions and Answers

Q1. What is Linux?

Linux is an open-source, Unix-like operating system kernel that manages hardware resources and provides services to applications. In common usage, Linux also refers to complete operating systems built around this kernel.

Q2. What is the difference between Linux and a Linux distribution?

Linux is the kernel, whereas a distribution combines the kernel with system utilities, libraries, package managers and applications to provide a complete operating system.

Q3. What is the main responsibility of the Linux kernel?

The kernel manages system resources such as CPU, memory, storage, devices and networking, and provides controlled access to these resources for applications.

Q4. What is the difference between Kernel Space and User Space?

User Space is where normal applications execute with restricted privileges. Kernel Space is where the kernel executes with privileged access to system resources.

Q5. What is a shell?

A shell is a command interpreter that accepts and executes user commands. Bash is one of the most commonly used Linux shells.

Q6. What is the difference between a shell and a terminal?

A terminal provides the interface for entering and displaying commands. A shell interprets and executes those commands.

Q7. What is a system call?

A system call is an interface through which an application requests services from the Linux kernel, such as reading files, creating processes or communicating over a network.

Q8. Is Linux a complete operating system or just a kernel?

Technically, Linux is the kernel. However, in everyday usage, Linux commonly refers to a complete operating system built around the Linux kernel.

Q9. Why is Linux widely used in DevOps?

Linux offers powerful command-line tools, scripting, automation, networking, security and compatibility with cloud and container technologies. These capabilities make it suitable for managing infrastructure and application deployments.

Q10. What happens when you execute `ls -l`?

The shell interprets the command and checks for aliases and built-ins, then searches PATH to locate the executable. It creates a child process using `fork()`, and the child runs `ls` using `execve()`. The `ls` program uses system calls to read directory information from the kernel, and the output is displayed in the terminal.

Q11. What is the difference between a process and a thread?

A process has its own memory space and PID. A thread runs inside a process and shares its memory. Threads are faster to create, but processes are better isolated from each other.

13 Chapter Takeaways

After completing this chapter, you should be able to explain:

- ✓ What Linux is and why it is used.
- ✓ Difference between Linux, Unix and Linux distributions.
- ✓ Linux architecture and its major components.
- ✓ What the Linux kernel does.
- ✓ Difference between Kernel Space and User Space.
- ✓ What system calls are and why applications need them.
- ✓ Difference between a terminal and a shell.
- ✓ How Linux commands are executed.
- ✓ Difference between a process, a thread and a daemon.
- ✓ What GNU/Linux means.
- ✓ Basic Linux system information commands.

IMPORTANT INTERVIEW REMINDER

Do not just memorize the definitions. For every concept, try to explain:

1. What it is.
2. Why it exists.
3. How it works.
4. Where you have seen it in real-world DevOps work.

END OF CHAPTER 1